Automatización y Calidad de Código

De scripts desechables a herramientas profesionales verificables.

```
> automatizacion_pathlib.py
> pytest_verificacion.py
> documentacion_pep257.md
```

Ciclo IV — Ingeniería de Sistemas
Cierre de la Unidad I

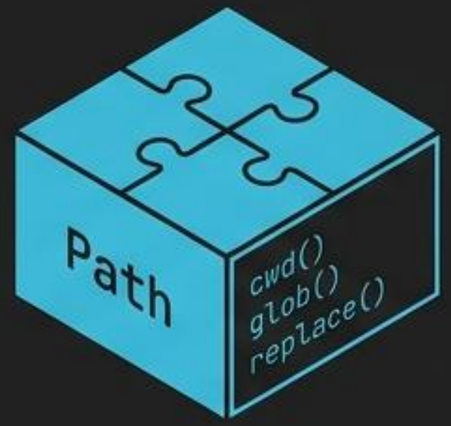
La Evolución: De os a pathlib

El Método Clásico - Disperso



Las rutas como cadenas de texto.
Obliga a mezclar módulos para tareas simples.

El Método Moderno - Unificado



Interfaz orientada a objetos consistente
entre sistemas operativos.

```python
from pathlib import Path

for archivo in Path.cwd().glob("*.txt"):
    destino = Path("archivo") / archivo.name
    archivo.replace(destino)
```

División del Trabajo: `pathlib` vs. `shutil`



`pathlib`

- Construir rutas
- Validar existencia
- Leer/escribir archivos pequeños
- Iterar carpetas



`shutil`

- Operaciones de alto nivel
- Copiar árboles completos
- Empaquetar archivos comprimidos
- Eliminación recursiva

```python
import shutil
from pathlib import Path

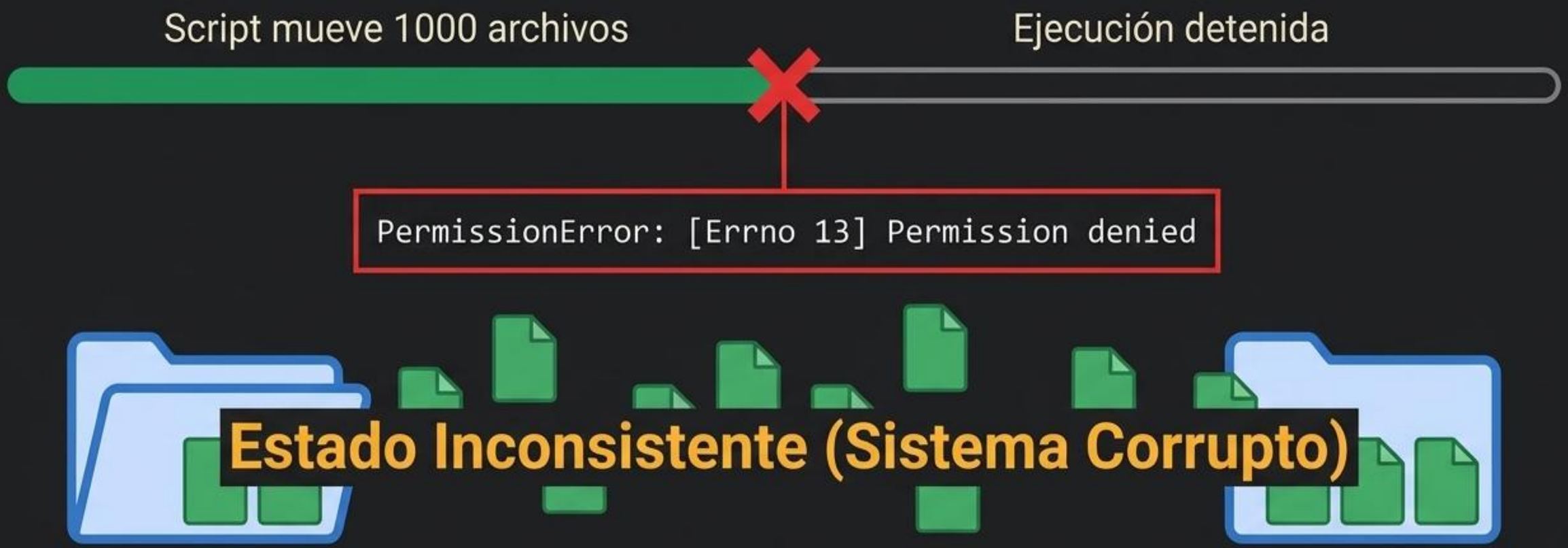
origen = Path("reportes")
shutil.make_archive("backup/reportes", "zip", root_dir=origen)
```

Anatomía de un Script Profesional



Esta combinación distingue un script desechable de un artefacto de software reutilizable.

El Peligro de los Efectos Destructivos



Un script de automatización que manipula el sistema de archivos es peligroso si falla. Regla profesional: Probar antes de confiar.

Introducción a pytest

- El framework de pruebas estándar de la industria moderna en Python.
- Rechaza el código verboso a favor de la simplicidad de la palabra clave nativa assert.

```python
def sumar(a, b):
    return a + b

def test_sumar():
    assert sumar(2, 3) == 5
```

Evaluación directa y legible. Si es False, la prueba falla y pytest reporta el error.

Fixtures y el Sandbox (tmp_path)

Fixture: Un contexto inyectado automáticamente por pytest que garantiza un entorno predecible y seguro.

Disco C: / OS Real



Intocable durante pruebas

tmp_path (Sandbox de pytest)



Entorno aislado temporal por prueba

```python
def test_organizar_crea_subcarpeta(tmp_path):
    archivo = tmp_path / "foto.jpg"
    archivo.write_text("contenido")

    organizar_carpeta(tmp_path)

    assert (tmp_path / "Imagenes" / "foto.jpg").exists()
```

Anatomía de una Prueba: AAA (Arrange, Act, Assert)

```python
def test_organizar_crea_subcarpeta(tmp_path):
    archivo = tmp_path / "foto.jpg"
    archivo.write_text("contenido")

    organizar_carpeta(tmp_path)

    assert (tmp_path / "Imagenes" / "foto.jpg").exists()
```

Arrange (Preparar): Configurar el contexto y los datos iniciales confiables.

Act (Actuar): Ejecutar la función bajo prueba.

Assert (Afirmar): Verificar que el estado resultante es el esperado.

⚠️ Nota: Si la fase de Arrange falla (ej. error en la fixture), pytest lo marca como Error, no como prueba fallida.

Ejecución y Reporte en Terminal

```
$ pytest                          # Ejecuta todas las pruebas descubiertas
$ pytest test_organizador.py      # Ejecuta un archivo específico
$ pytest -v                       # Modo detallado / verbose

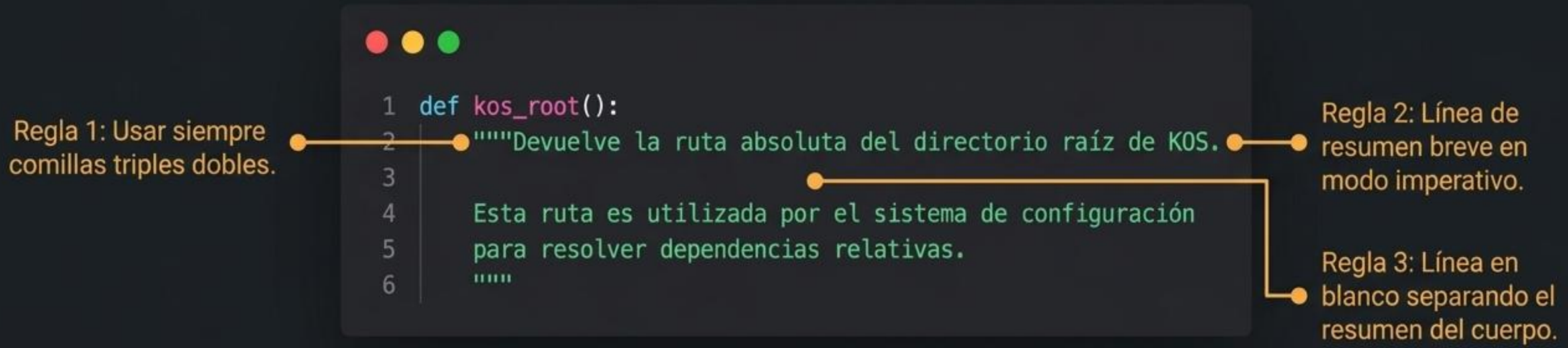
========================== test session starts ==========================
collected 2 items

test_organizador.py::test_organizar_crea_subcarpeta      PASSED  [ 50%]
test_organizador.py::test_organizar_ignora_ocultos       PASSED  [100%]

============================ 2 passed in 0.04s ============================
```

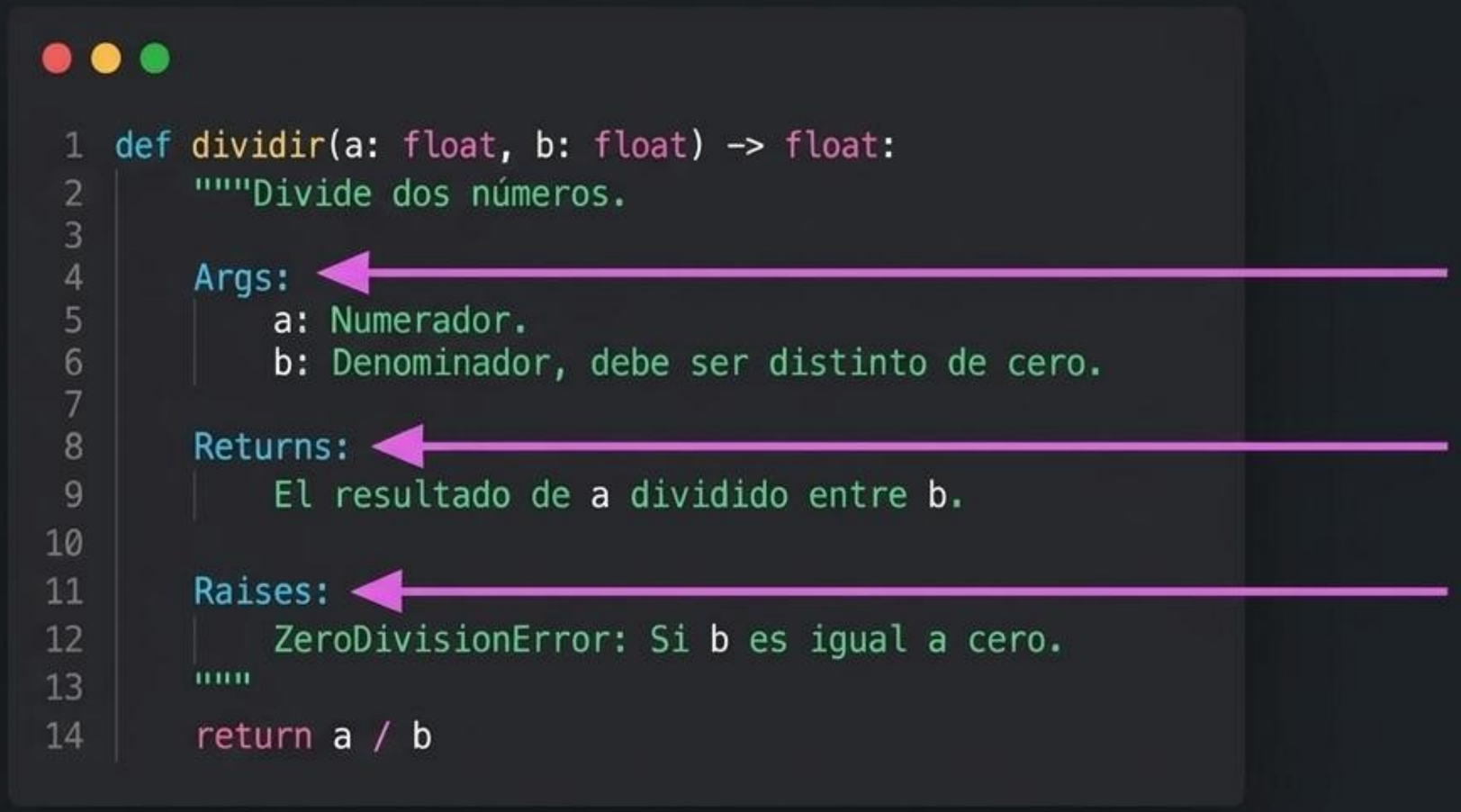
Convenciones Base: PEP 257

El código se lee más veces de las que se escribe. PEP 257 estandariza qué deben contener los docstrings y cómo estructurarlos.



Estilo Google para Docstrings

La estructura más adoptada en la industria. Compatible con generadores automáticos como Sphinx.



Type Hints: Documentación Verificable

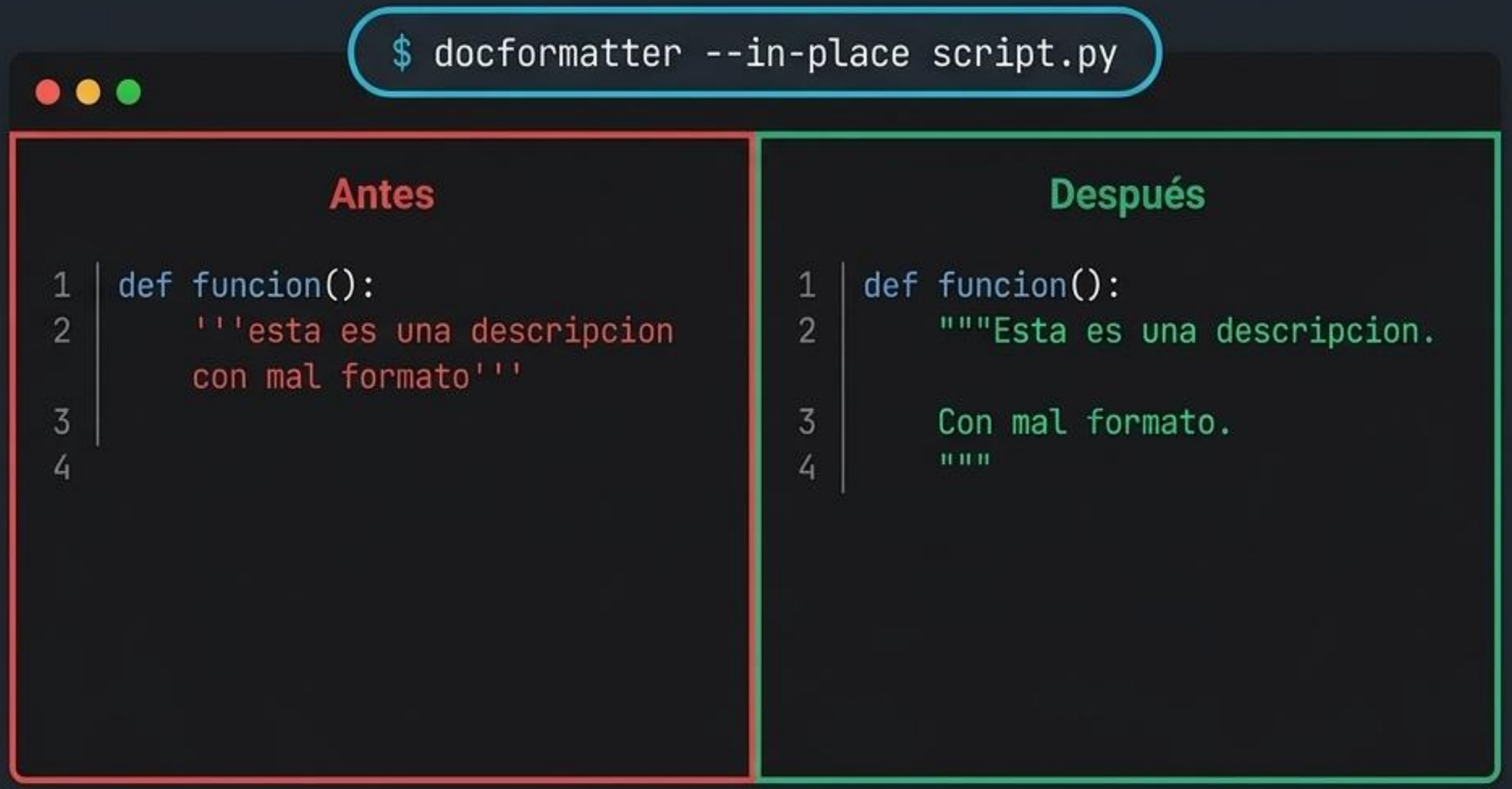
```python
def dividir(a: float, b: float) -> float:
```

- **El Problema:** Los comentarios de texto pueden quedar desactualizados respecto al código.
- **La Solución:** Al usar Type Hints (PEP 484), el intérprete y el IDE conocen el tipo de dato real.

Insight Clave: La guía de estilo Google indica que **no es necesario especificar el tipo de dato dentro del bloque Args:** si ya existe un type hint. La firma de la función actúa como **documentación validable por máquina.**

Automatización del Estilo: docformatter

Así como usamos Black para el estilo del código, usamos **docformatter** para forzar el cumplimiento de PEP 257.



Entregable Unidad I: Herramienta de Automatización

Motor Lógico (pathlib + shutil)

Automatiza una tarea de archivos (ej. organizador por lotes).

Garantía de Calidad (pytest)

Mínimo 2 pruebas automatizadas usando la fixture `tmp_path`.



Especificación Técnica (PEP 257)

Docstring de módulo y funciones en Estilo Google con Type Hints.

Este entregable demuestra el dominio del ciclo completo: construir, proteger y documentar.

Referencias y Bibliografía

- Percival, H. & Gregory, B. (2020). Test-Driven Development with Python, 2.ª ed. O'Reilly.
- Lutz, M. (2013). Learning Python, 5.ª ed. O'Reilly.
- pytest documentation (2026). About fixtures. docs.pytest.org
- Python documentation (2026). pathlib & shutil. docs.python.org
- Google Python Style Guide (2026).
- PEP 257 – Docstring Conventions. peps.python.org

- Python, H. & Gregory, (2026). Test-ltbb & shutil. docs.python.org

- Google Python Style Guide (2026).

- PEP 257 – Docstring Conventions. peps.python.org